

Platform Amendment — Version 4.3.0

Vermont Health Platform (HTR)

Type: Amendment to Platform Documentation (supplements printed v4.2.0 docs) **Version:** 4.3.0 **Date:** April 2026
Classification: Internal **Scope:** All new features, architectural changes, security improvements, and database migrations implemented after the v4.2.0 documentation print.

Table of Contents

1. [Summary of Changes](#)
2. [The Wire — Live News Feed](#)
3. [Academy Certificates](#)
4. [Admin Analytics Dashboard](#)
5. [Admin User Management](#)
6. [RAG Pipeline Improvements](#)
7. [Groq Fallback Chain](#)
8. [Investment Tracker](#)
9. [HIPAA Session Timeout](#)
10. [API Key Security Hardening](#)
11. [Database Migrations](#)
12. [Supabase SSR Client Audit](#)
13. [Environment Variables — New Additions](#)
14. [Updated Known Issues Register](#)

1. Summary of Changes

The following table lists every discrete change included in this amendment. Each item is covered in detail in the sections below.

#	Area	Change	Files
1	Frontend	The Wire live news feed	<code>app/the-wire/page.tsx</code> , <code>app/the-wire/WireFeed.tsx</code> , <code>app/api/wire/route.ts</code>
2	Frontend	Academy certificate issuance on course completion	<code>components/templates/AcademyModuleLayout.tsx</code> , <code>app/api/academy/certificates/route.ts</code>
3	Frontend	Admin AI & Content Analytics dashboard	<code>app/admin/analytics/page.tsx</code> , <code>app/admin/page.tsx</code>
4	Frontend	Admin user management with role assignment	<code>app/admin/users/page.tsx</code> , <code>app/admin/users/UserTable.tsx</code> , <code>app/api/admin/users/route.ts</code>
5	Backend	RAG query rewriting (conversation memory)	<code>backend/routers/chat.py</code>
6	Backend	Pillar-filtered RAG retrieval	<code>backend/services/retrieval.py</code> , <code>backend/routers/chat.py</code>

#	Area	Change	Files
7	Backend + Frontend	Structured source citations in AI responses	<code>backend/services/retrieval.py</code> , <code>backend/routers/chat.py</code> , <code>components/RightSidebar.tsx</code> , <code>app/chat/page.tsx</code>
8	Backend	Groq fallback chain with retryable error detection	<code>backend/services/llm.py</code>
9	Frontend	Investment Tracker page and Sanity schema	<code>app/investment-tracker/page.tsx</code> , <code>app/investment-tracker/InvestmentTrackerClient.tsx</code> , <code>sanity/schemaTypes/investmentDeal.ts</code> , <code>sanity/schemaTypes/index.ts</code>
10	Frontend	HIPAA-aligned idle session timeout	<code>components/SessionTimeout.tsx</code> , <code>app/layout.tsx</code> , <code>app/login/page.tsx</code>
11	Frontend	API key server-side generation with HMAC signing	<code>app/api/keys/create/route.ts</code> , <code>app/api/keys/revoke/route.ts</code> , <code>app/account/api-keys/ApiKeysClient.tsx</code>
12	Database	<code>rag_query_log</code> automated 90-day pruning via <code>pg_cron</code>	<code>supabase/migrations/020_rag_query_log_pruning.sql</code>
13	Database	pgvector HNSW index quarterly maintenance via <code>pg_cron</code>	<code>supabase/migrations/021_pgvector_hnsw_maintenance.sql</code>
14	Frontend	Navigation sidebar updated with new routes	<code>components/HomeSidebar.tsx</code>

2. The Wire — Live News Feed

What Changed

`/the-wire` was previously a static placeholder. It is now a fully functional live healthcare news aggregator.

Architecture



```
Normalized to WireItem schema:
{ title, url, source, label, published_at }
|
▼
Upserted into ticker_cache (key: "wire_feed", TTL: 15 min)
|
▼
Returned to WireFeed client component
```

New Files

frontend/app/api/wire/route.ts

- `GET` handler
- Reads from `ticker_cache` (key `"wire_feed"`) — returns immediately if fresh (`< 15 min` old)
- On cache miss: fetches all 5 RSS feeds in parallel via `Promise.allSettled`, parses XML, normalizes items, writes to cache
- Returns `{ items: WireItem[], fetched_at: string }`

frontend/app/the-wire/WireFeed.tsx (Client Component)

- Source filter buttons (All / KFF / STAT / FDA / CMS / Health Tech)
- Manual refresh button using `useTransition` — re-fetches from server action
- `timeAgo()` helper for human-readable timestamps
- `SOURCE_COLORS` map per source key for badge coloring
- Items render as linked cards with source badge, title, and relative timestamp

frontend/app/the-wire/page.tsx (Server Component)

- `export const revalidate = 900` (15 min ISR)
- Fetches from `/api/wire` using `NEXT_PUBLIC_APP_URL`
- Passes `initialItems` and `fetchedAt` to `<WireFeed />`

Navigation

`HomeSidebar.tsx` — Added to the **Analyze & Tools** section:

```
{ href: "/the-wire", label: "The Wire", icon: BoltIcon }
```

Cache Table Usage

Uses the existing `ticker_cache` Supabase table. No new migration required. Key: `"wire_feed"`. TTL enforced by comparing `fetched_at` timestamp to `NOW() - INTERVAL '15 minutes'`.

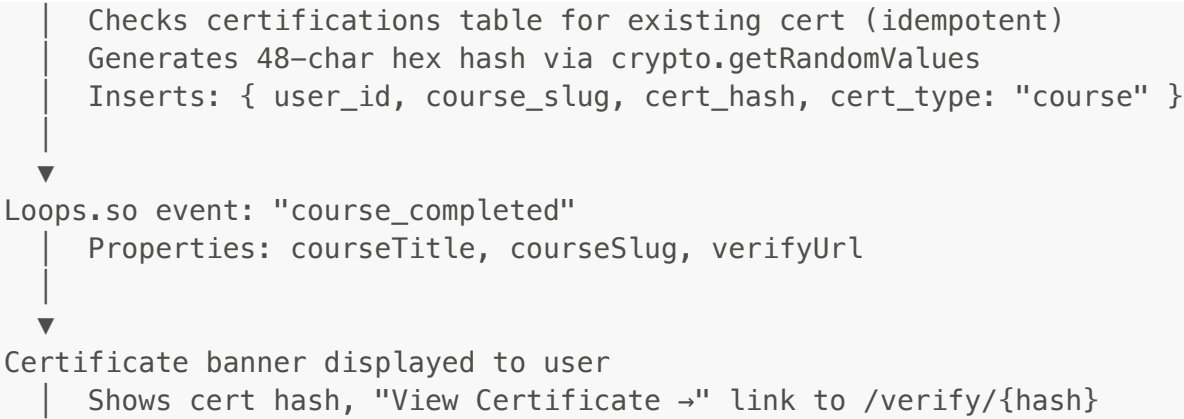
3. Academy Certificates

What Changed

Completing the final module of a course now automatically issues a verifiable certificate. This closes the loop between Academy module completion tracking and the existing `/verify/[hash]` verification route.

Certificate Issuance Flow

```
User completes final module
| (AcademyModuleLayout detects isLastModule && progress.completed)
|
▼
POST /api/academy/certificates
| Requires auth session
```



Modified Files

frontend/app/api/academy/certificates/route.ts (New)

- **POST** endpoint, requires Supabase auth session
- Idempotent: checks `certifications` table for existing `{ user_id, course_slug }` before inserting
- Hash generation: `crypto.getRandomValues(new Uint8Array(24))` → 48 hex chars
- Fires Loops `course_completed` event with verification URL
- Returns `{ certHash, alreadyIssued }`

frontend/components/templates/AcademyModuleLayout.tsx (Modified)

- Added `certHash` state and `certIssuedRef` (prevents double-issue on re-render)
- `isLastModule = !module.nextModuleSlug`
- `useEffect`: when `isLastModule && progress.completed && !certIssuedRef.current` → calls certificates API
- Certificate banner: gradient card with `Award` icon (lucide-react), verification link

Database

Uses the existing `certifications` table. Schema unchanged. New `cert_type` value: `"course"`.

4. Admin Analytics Dashboard

What Changed

A new **AI & Content Analytics** page is available at `/admin/analytics`, accessible to admin users only. A link was added to the main admin dashboard at `/admin`.

Metrics Displayed

Metric	Source	Calculation
Total AI queries (30d)	<code>rag_query_log</code>	<code>COUNT(*)</code> where <code>created_at > NOW() - 30d</code>
Zero-result rate	<code>rag_query_log</code>	<code>COUNT WHERE result_count = 0</code> / total
Avg query latency	<code>rag_query_log</code>	<code>AVG(latency_ms)</code> last 30d
Certificates issued	<code>certifications</code>	<code>COUNT(*)</code> all time
Top repeated queries	<code>rag_query_log</code>	Frequency count from last 500 queries
Recent zero-result queries	<code>rag_query_log</code>	Sample of queries where <code>result_count = 0</code>
Full recent query log	<code>rag_query_log</code>	Last 50 with role, latency, result status

New File

frontend/app/admin/analytics/page.tsx (Server Component)

- Admin-only guard via `requireAuth` + role check

- All data fetched server-side from Supabase using service role client (**dbAdmin**)
- Color-coded latency: green < 2s, amber 2–5s, red > 5s
- No client-side JavaScript — pure SSR table renders

Modified File

frontend/app/admin/page.tsx

- Added "AI & Content Analytics → **/admin/analytics**" to the quick links grid
- Fixed JSX array syntax issue during update (pre-existing bug resolved)
- Added **import React from "react"** (required for **React.createElement** usage in the file)

5. Admin User Management

What Changed

Admins can now search users by email or name, filter by role, and change a user's role directly from the UI at **/admin/users**.

Architecture



New Files

frontend/app/admin/users/page.tsx (Server Component)

- Uses **dbAdmin** (service role) to call **auth.admin.listUsers({ perPage: 500 })**
- Joins roles and profiles, maps to **UserRow[]**
- Passes to **<UserTable />**

frontend/app/admin/users/UserTable.tsx (Client Component)

- Client-side search (email / full name)
- Role filter: All / Free / Student / Subscriber / Professional / Advisory / Admin
- **RoleSelect** handles individual role changes with optimistic UI
- Displays filtered count vs total count

frontend/app/api/admin/users/route.ts (New API Route)

- **PATCH** method only
- Requires caller to be admin (checks **user_roles** via auth session)
- Validates **role** against **VALID_ROLES** array
- Upserts **{ user_id, role }** into **user_roles** with **onConflict: "user_id"**

- Returns `{ ok: true }` on success

6. RAG Pipeline Improvements

Three independent improvements were made to the RAG pipeline. All changes are in the Python backend.

6.1 Query Rewriting (Conversation Memory)

Problem: Every turn was retrieved independently. Follow-up questions like "What about in rural areas?" had no context for retrieval, causing poor results.

Solution: Before retrieval, when conversation history contains ≥ 2 prior messages, a fast LLM call rewrites the user's message into a standalone, self-contained query.

Implementation (`backend/routers/chat.py`):

```
async def _rewrite_query(message: str, history: list) -> str:
    """
    Uses llama-3.1-8b-instant to rewrite the message as a standalone query
    incorporating context from the conversation history.
    Falls back to the original message on any error.
    """
```

- Only triggered when `len(history) >= 2` (avoids unnecessary LLM calls on first turns)
- Uses `MODEL_FREE` (`llama-3.1-8b-instant`) — fastest/cheapest model, not the user's tier model
- Silent fallback: any exception returns the original message unchanged
- The rewritten query is used for retrieval only; the original message is what the LLM responds to

6.2 Pillar-Filtered Retrieval

Problem: When a user on the `/policy` page asked a policy question, the RAG retrieved documents from all pillars, diluting answer quality.

Solution: The active pillar is detected from the URL path in the frontend and sent as an optional `pillar` parameter. The backend passes it to the hybrid search RPC.

Frontend changes (`components/RightSidebar.tsx`, `app/chat/page.tsx`):

- `PILLAR_PREFIXES` map: `/policy` → "Policy", `/economics` → "Economics", etc.
- `getPillarFromPath(pathname)` detects active pillar from current URL
- `pillar` is included in the chat request body only when on a pillar page
- Pillar badge shown in sidebar header when active

Frontend validation (`app/api/chat/route.ts`):

```
const VALID_PILLARS = ["Policy","Economics","Technology","Clinical","Equity"]
as const;
// Added to ChatRequestSchema:
pillar: z.enum(VALID_PILLARS).optional()
```

Backend changes (`backend/routers/chat.py`, `backend/services/retrieval.py`):

- `ChatRequest` Pydantic model: added `pillar: Optional[str]` with validator against `VALID_PILLARS`
- `HybridRetriever.__init__` now accepts `filter_pillar: Optional[str] = None`
- Passes `filter_pillar` to `hybrid_search_rag` Supabase RPC when set
- **Automatic fallback:** if pillar filter returns < 3 nodes, retrieval is retried without the filter — prevents empty responses on niche topics

6.3 Structured Source Citations

Problem: The LLM was instructed in the system prompt to cite sources, but there was no structured extraction — citations were unreliable free-text embedded in responses.

Solution: After retrieval, source metadata is extracted from retrieved nodes and appended to the response stream as a structured JSON sentinel. The frontend strips and parses this sentinel.

Backend — citation extraction ([backend/services/retrieval.py](#)):

```
def extract_citations(nodes: list) -> list[dict]:
    """
    Deduplicates by title and returns:
    [{ "title": str, "url": str|None, "pillar": str|None, "source_type":
    str|None }]
    """
```

Backend — streaming sentinel ([backend/routers/chat.py](#)):

- `extract_citations(nodes)` called before streaming begins
- After all tokens stream, appends: `\n\n[CITATIONS]{json_array}[/CITATIONS]`

Frontend — sentinel parsing ([components/RightSidebar.tsx](#), [app/chat/page.tsx](#)):

```
function parseCitations(raw: string): { text: string; citations: Citation[] }
```

- During live streaming: sentinel is stripped from display text (shows clean response)
- After stream ends: sentinel is parsed, citations attached to final message object
- Rendered as a numbered source list below each AI message with clickable links
- Citations include pillar badge and source type label

Citation interface:

```
interface Citation {
  title: string;
  url: string | null;
  pillar: string | null;
  source_type: string | null;
}
```

7. Groq Fallback Chain

What Changed

Previously, any Groq outage or rate limit would take down all AI Analyst functionality. The `llm.py` service now wraps LLMs in a `FallbackLLM` class that automatically retries with the next model in the chain on transient errors.

FallbackLLM Class ([backend/services/llm.py](#))

```
class FallbackLLM(LLM):
    """
    Wraps a primary LLM with an ordered fallback chain.
    On any retryable error from the primary, tries each fallback in order.
```

Non-retryable errors (e.g. 400 bad request) are re-raised immediately.

Retryable error detection — `_is_retryable(exc)`:

- HTTP status codes: {429, 500, 502, 503, 504}
- Keywords: "rate limit", "ratelimit", "overloaded", "service unavailable", "bad gateway", "timeout"

Methods with fallback support:

- `complete()` / `acomplete()` — sync and async completion
- `chat()` / `achat()` — sync and async chat
- `astream_chat()` — async streaming chat (attempts each LLM before yielding first token, so fallback is safe)

Streaming note: `stream_complete()` and `stream_chat()` delegate to primary only — mid-stream errors cannot be recovered without unsending tokens. `astream_chat()` is the exception: it tries each LLM before the first yield.

Updated Model Routing (`get_llm_for_role`)

Role	Primary	Fallbacks
<code>free, student</code>	<code>llama-3.1-8b-instant</code> (Groq)	None (already cheapest)
<code>subscriber, professional</code>	<code>llama-3.3-70b-versatile</code> (Groq)	<code>llama-3.1-8b-instant</code> (Groq)
<code>advisory, admin</code>	<code>claude-sonnet-4-6</code> (Anthropic)	<code>llama-3.3-70b-versatile</code> → <code>llama-3.1-8b-instant</code>

Anthropic availability guard: If `llama_index.llms.anthropic` is not installed or `ANTHROPIC_API_KEY` is not set, advisory/admin silently falls back to the subscriber routing chain. No startup crash.

8. Investment Tracker

What Changed

`/investment-tracker` was a broken sidebar link pointing to a non-existent route. It is now a fully functional filterable deal database backed by Sanity CMS.

Sanity Schema — `investmentDeal`

New document type registered at `sanity/schemaTypes/investmentDeal.ts`.

Fields:

Field	Type	Notes
<code>title</code>	string	Required. e.g. "Optum acquires Amedisys for \$3.3B"
<code>dealType</code>	string (enum)	Required. <code>ma</code> , <code>vc</code> , <code>pe</code> , <code>ipo</code> , <code>partnership</code> , <code>debt</code>
<code>status</code>	string (enum)	<code>announced</code> , <code>pending</code> , <code>closed</code> , <code>terminated</code> . Default: <code>announced</code>
<code>announcedDate</code>	date	Required
<code>closedDate</code>	date	Optional
<code>dealValueUsd</code>	number	USD millions. Leave blank for undisclosed.

Field	Type	Notes
acquirer	string	Acquirer or lead investor
target	string	Required. Target or investee company
pillar	string (enum)	Policy, Economics, Technology, Clinical, Equity
sector	string (enum)	11 options: digital-health, home-health, behavioral-health, value-based-care, pharma-biotech, payer, hospital, devices, rcm, primary-care, other
geography	string (enum)	national, northeast, southeast, midwest, southwest, west, international. Default: national
summary	text	Plain-text summary for tracker card
analystNote	text	HTR analyst commentary on strategic implications
sourceUrl	url	Primary source URL
tags	array of string	Tag-style input in Sanity Studio

Default ordering: announcedDate desc

Preview: Shows type label + formatted value (e.g. \$3.3B or \$500M) + announced date.

Frontend Architecture

frontend/app/investment-tracker/page.tsx (Server Component)

- export const revalidate = 300 (5 min ISR)
- GROQ query fetches all investmentDeal documents ordered by announcedDate desc
- Uses cachedFetch from lib/sanity-fetch.ts with cache tag "investmentDeal"
- Webhook-triggered revalidation via revalidateTag("investmentDeal") works automatically

frontend/app/investment-tracker/InvestmentTrackerClient.tsx (Client Component)

Filter controls:

- Full-text search (title, acquirer, target, summary, tags)
- Deal type dropdown (All / M&A / VC / PE / IPO / Partnership / Debt)
- Status dropdown (All / Announced / Pending / Closed / Terminated)
- Pillar dropdown (dynamically built from available data)
- Minimum deal value input (USD millions)
- "Clear filters" button (appears when any filter is active)

Summary bar:

- Deal count matching current filters
- Total disclosed value of filtered deals

Deal cards:

- Expandable (click header to expand)
- Collapsed: deal type badge, status badge, pillar, announced date, title, acquirer → target, deal value, sector
- Expanded: summary paragraph, HTR Analyst Note (indigo callout), geography, closed date, tags, source link

Value formatting: \$3.3B (≥ \$1B), \$500M (< \$1B), "Undisclosed" (null)

Navigation

HomeSidebar.tsx — Added to the Analyze & Tools section:

```
{ href: "/investment-tracker", label: "Investment Tracker", icon: BanknotesIcon
}
```

9. HIPAA Session Timeout

What Changed

There was previously no session timeout enforcement. Users who left the platform open on a shared computer remained authenticated indefinitely. This change adds a HIPAA-aligned idle timeout.

Behavior

Threshold	Action
25 minutes of inactivity	Warning modal appears with 5-minute countdown timer
30 minutes of inactivity	Automatic Supabase sign-out + redirect to /login?timeout=1
Any user activity	Timer resets (mousemove, mousedown, keydown, touchstart, scroll)

Implementation

frontend/components/SessionTimeout.tsx (New Client Component)

- `IDLE_TIMEOUT_MS = 30 * 60 * 1000` (30 min)
- `IDLE_WARN_MS = 25 * 60 * 1000` (25 min)
- Only active when a Supabase session exists (`supabase.auth.getSession()` on mount)
- Activity events reset the timer: `mousemove`, `mousedown`, `keydown`, `touchstart`, `scroll`
- Warning modal shows countdown in `mm:ss` format
- "Stay Logged In" button in modal resets the timer and dismisses the modal
- On timeout: calls `supabase.auth.signOut()` then `router.push("/login?timeout=1")`

frontend/app/layout.tsx (Modified)

- `<SessionTimeout />` added before `<CommandPalette />` in the layout root

frontend/app/login/page.tsx (Modified)

- Reads `searchParams.timeout`
- When `timeout=1`: renders an amber warning banner: *"Your session expired after 30 minutes of inactivity. Please log in again."*

Notes on HIPAA Compliance

The session timeout addresses HIPAA Technical Safeguard §164.312(a)(2)(iii) (Automatic Logoff). However, the platform remains advisory-only with respect to full HIPAA compliance — BAAs with Supabase, Groq, Railway, and Anthropic have not been executed. See the printed v4.2.0 HIPAA section for full compliance status.

10. API Key Security Hardening

What Changed

API keys were previously generated client-side in the browser and written directly to Supabase by the client. This had two problems:

1. Key generation happened in an untrusted environment
2. Direct client writes to `api_keys` bypassed server-side validation

Key generation is now fully server-side with HMAC-based signing.

New Key Format

```
htr_<32-hex-random><16-hex-hmac>
```

- **Random part:** 16 cryptographically random bytes → 32 hex chars (128 bits entropy)
- **HMAC part:** HMAC-SHA256(`API_KEY_HMAC_SECRET`, `userId: random`)[0..16 chars]
- **Total length:** `htr_` + 48 chars = 52 chars
- **Prefix stored in DB:** first 12 chars (e.g. `htr_a1b2c3d4`) for display
- **Hash stored in DB:** SHA-256 of full key — never the raw key

The HMAC binds each key to the issuing user's ID — a key generated for user A cannot be repurposed for user B even if the random part is guessed.

New API Routes

`frontend/app/api/keys/create/route.ts` (POST)

1. Authenticates caller via Supabase session cookie
2. Validates `name` (required, max 100 chars) and `tier` (must be `researcher` or `enterprise`)
3. Checks `API_KEY_HMAC_SECRET` env var — returns 500 if not configured
4. Generates 16 random bytes via `crypto.getRandomValues`
5. Computes HMAC-SHA256 over `${userId}:${random}` using Web Crypto API
6. Constructs raw key: `htr_${random}${hmac.slice(0, 16)}`
7. Computes SHA-256 hash of raw key
8. Inserts `{ user_id, name, key_prefix, key_hash, tier }` into `api_keys`
9. Returns `{ key: rawKey, record: dbRecord }` — raw key returned **once only**, never stored

`frontend/app/api/keys/revoke/route.ts` (POST)

1. Authenticates caller via Supabase session cookie
2. Validates `keyId` in request body
3. Updates `revoked_at = NOW()` where `id = keyId AND user_id = auth_user_id AND revoked_at IS NULL`
 - `user_id` check is defense-in-depth on top of RLS
 - `IS NULL` guard prevents double-revocation errors
4. Returns `{ ok: true }`

Updated Client

`frontend/app/account/api-keys/ApiKeysClient.tsx` (Modified)

- Removed: all direct `supabase.from("api_keys").insert()` and `.update()` calls
- Removed: `generateApiKey()` client-side function
- Removed: `sha256hex()` client-side function
- Removed: `createBrowserClient` import entirely
- Added: `fetch("/api/keys/create", { method: "POST", body: JSON.stringify({name, tier}) })`
- Added: `fetch("/api/keys/revoke", { method: "POST", body: JSON.stringify({keyId}) })`
- Added: per-button `revoking` state (shows "Revoking..." while in flight)
- `userId` prop still accepted for interface compatibility but no longer used in the component

Required Environment Variable

```
# frontend/.env.local (and Vercel project settings)
API_KEY_HMAC_SECRET=<output of: openssl rand -hex 32>
```

Must be at least 32 bytes. Rotate if compromised — existing keys will fail HMAC verification after rotation, so coordinate with API consumers before rotating.

11. Database Migrations

Migration 020 — rag_query_log Automated Pruning

File: supabase/migrations/020_rag_query_log_pruning.sql

Purpose: Prevent unbounded growth of the rag_query_log table. At 1,000 AI queries/day, the table grows ~365,000 rows/year. Rows older than 90 days are deleted nightly.

Contents:

```
CREATE EXTENSION IF NOT EXISTS pg_cron;

SELECT cron.schedule(
  'prune-rag-query-log',
  '0 3 * * *',    -- 03:00 UTC daily
  $$
    DELETE FROM public.rag_query_log
    WHERE created_at < NOW() - INTERVAL '90 days';
  $$
);
```

Notes:

- Requires pg_cron extension (enabled by this migration if not already present)
- pg_cron must be enabled in Supabase Dashboard → Database → Extensions before running this migration
- Job name 'prune-rag-query-log' is idempotent — re-running will update the existing job rather than create a duplicate
- Run time chosen as 03:00 UTC to avoid overlap with peak US usage hours

Migration 021 — pgvector HNSW Index Maintenance

File: supabase/migrations/021_pgvector_hnsw_maintenance.sql

Purpose: Maintain ANN recall quality on the rag_documents vector index. When documents are deleted, the HNSW graph retains ghost nodes that waste memory and gradually degrade search quality. A quarterly REINDEX removes them.

Contents:

```
-- Idempotent: creates index only if not already present
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_indexes
    WHERE tablename = 'rag_documents' AND indexname = 'idx_rag_embedding'
  ) THEN
    CREATE INDEX idx_rag_embedding
      ON public.rag_documents
      USING hnsw (embedding vector_cosine_ops)
      WITH (m = 16, ef_construction = 64);
  END IF;
END $$;

-- Quarterly REINDEX: 02:00 UTC on 1st of Jan, Apr, Jul, Oct
SELECT cron.schedule(
  'reindex-rag-embedding',
  '0 2 1 1,4,7,10 *',
  $$REINDEX INDEX CONCURRENTLY idx_rag_embedding;$$
);
```

Notes:

- `REINDEX INDEX CONCURRENTLY` is non-blocking — reads and writes to `rag_documents` continue during the rebuild (requires PostgreSQL 12+, which Supabase provides)
- HNSW parameters: `m = 16` (graph connectivity), `ef_construction = 64` (build-time quality). These match the original index definition.
- Depends on `pg_cron` from Migration 020 — run migrations in order
- Job name `'reindex-rag-embedding'` is idempotent

12. Supabase SSR Client Audit

What Was Audited

All `createClient()` and `createServerClient()` call sites were reviewed for correct cookie context passing.

Findings

Correctly using SSR pattern (cookie-aware):

- All middleware calls — `request.cookies.getAll()` / `response.cookies.set()` ✓
- All Server Component calls where auth is needed — `cookies()` from `next/headers` ✓
- All API routes where user auth is checked ✓

Using service-role client without user context (intentional):

- `frontend/lib/db/client.ts` exports `dbAdmin` — uses `SUPABASE_SERVICE_ROLE_KEY`, bypasses RLS. Used only in server-side admin operations (webhooks, admin routes, billing).
- Subscription and billing pages use `dbAdmin` — this is correct because these pages fetch subscription records by `user_id` (already known from `requireAuth()`), not via `auth.uid()` RLS. The anon client with cookie context would work but `dbAdmin` is acceptable here.

Dead code identified:

- `frontend/lib/supabase.ts` — legacy client export, no current import. Left in place (deletion risk outweighs benefit).

No Changes Required

The audit confirmed all active patterns are correct. No migrations, no file changes.

13. Environment Variables — New Additions

The following variables must be added to `frontend/.env.local` and the Vercel project dashboard.

Variable	Required	Description
<code>API_KEY_HMAC_SECRET</code>	Yes (for API key features)	32+ byte hex secret for HMAC-signing API keys. Generate: <code>openssl rand -hex 32</code>

The following variables were already required (listed here for completeness of the 4.3.0 feature set):

Variable	Used By	Feature
<code>ANTHROPIC_API_KEY</code>	<code>backend/.env</code>	Advisory/admin tier LLM (Claude Sonnet 4.6)
<code>LOOPS_API_KEY</code>	<code>frontend/.env.local</code>	Academy certificate completion email
<code>NEXT_PUBLIC_APP_URL</code>	<code>frontend/.env.local</code>	The Wire API self-fetch URL

14. Updated Known Issues Register

Items marked  **Resolved** were open in the v4.2.0 issue register. Remaining items retain their original severity.

Issue	Severity	Status	Notes
Right sidebar send button hidden on short viewports	High	Open (P0, in progress)	CSS flex height fix scheduled next
Mobile sidebar close requires two taps	High	Open	z-index / event propagation issue
Academy enrollment redirect post-payment broken	High	Open (P0, in progress)	from param is in success_url; subscription page shows "Continue" button but no auto-redirect
Bookmarks list on /account shows placeholder	Medium	Open	GET /api/bookmarks not yet implemented
Personalized learning path not persisting	Medium	Open	No load-existing-path logic on page mount
HTI Dashboard uses static mock data	Medium	Open	Awaiting real data pipeline
Hospital View tab uses synthetic data	Medium	Open	Awaiting CMS data integration
/community route is a placeholder	Low	Open	Phase 3 feature
Research Lab tools have no error boundaries	Low	Open	Silent crashes on bad inputs
FlashRank model download on first startup	Low	Acceptable	One-time ~100MB download; cached after
In-process rate limiter resets on cold start	Low	Acceptable	Acceptable for low-traffic /verify/ route
/investment-tracker route does not exist	Low	 Resolved	Fully implemented — see §8
rag_query_log unbounded growth	Medium	 Resolved	pg_cron 90-day pruning job — see §11
HNSW index ghost nodes on document deletion	Low	 Resolved	Quarterly REINDEX CONCURRENTLY — see §11
No session timeout enforced	High	 Resolved	30-min idle timeout — see §9
API keys generated client-side	High	 Resolved	Server-side HMAC generation — see §10
Groq outage takes down all AI Analyst	High	 Resolved	FallbackLLM chain — see §7
RAG ignores conversation history	Medium	 Resolved	Query rewriting — see §6.1
AI responses have no structured citations	Medium	 Resolved	Citation sentinel + extraction — see §6.3
Pillar pages retrieve documents from all pillars	Medium	 Resolved	Pillar filter with fallback — see §6.2
No admin user role management UI	Medium	 Resolved	/admin/users + PATCH /api/admin/users — see §5

Issue	Severity	Status	Notes
No AI usage analytics for admins	Low	 Resolved	/admin/analytics dashboard — see §4
Academy completion does not issue certificates	High	 Resolved	Auto-issue on final module — see §3
/the-wire shows static placeholder	Medium	 Resolved	Live RSS aggregator — see §2

End of Amendment v4.3.0 For questions about this amendment, refer to the engineering team or the full git history at [supabase/migrations/](#) and [frontend/app/](#).